MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 2: DIGITAL LOGIC SYSTEM

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 18 MARCH 2026

TABLE OF CONTENTS

1.0 INTRODUCTION

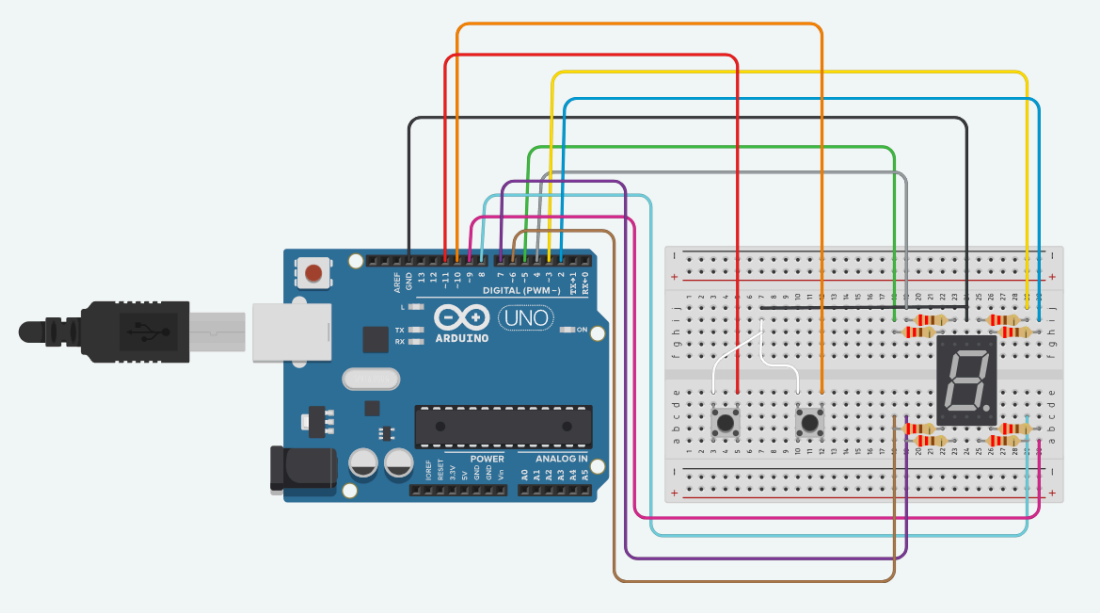
In this experiment, an Arduino Uno microcontroller was used to control a common cathode configuration of a 7-segment display. The display was controlled using push buttons. The display was controlled through the use of push buttons. One push button was configured to increment the number displayed on the 7-segment display, and the other push button was configured to reset the number displayed to 0. This experiment demonstrated that the digital output of a microcontroller can be used to control components external to the microcontroller while also receiving input signals through the use of push buttons.

The goal of this experiment is to demonstrate how to interface an Arduino microcontroller with a 7-segment display and also to observe how input and output operations of a digital microcontroller work to implement a simple digital logic system.

2.0 MATERIALS AND EQUIPMENT

- Arduino Uno board
- Common cathode 7-segment display
- 220-ohm resistors (7 of them)
- Pushbuttons (2 or more)
- Jumper wires
- Breadboard

3.0 EXPERIMENTAL SETUP



1. Connect the common cathode 7-segment display to the Arduino Uno as follows:

- Connect each of the 7 segments (a, b, c, d, e, f, g) of the display to separate digital pins on the Arduino (e.g., D0 to D6).
- Connect the common cathode pin of the display to one of the GND (ground) pins on the Arduino.
- Use 220-ohm resistors to connect each of the segment pins to the Arduino pins to limit the current.

2. Connect the pushbuttons to the Arduino:

- Connect one leg of each pushbutton to a separate digital pin (e.g., D9 and D10) and connect the other leg of each pushbutton to GND.
- Use 10K-ohm pull-up resistors for each pushbutton by connecting one end of each resistor to the digital pin and the other end to the 5V output of the Arduino.

4.0 METHODOLOGY

4.1 Circuit Setup

- All required components including the Arduino Uno, 7-segment display, resistors, push buttons, breadboard, and jumper wires were prepared before assembling the circuit.
- The 7-segment display segments (a–g) were connected to the digital pins of the Arduino through 220 Ω resistors to limit the current and protect the LED segments.
- The common cathode pin of the display was connected to the ground (GND) terminal of the Arduino board.
- Push buttons were connected to designated Arduino digital pins to serve as increment and reset inputs.
- 10 kΩ pull-up resistors were used to stabilize the input signals and prevent floating input states.
- After completing the connections, the wiring was checked carefully to ensure correct connectivity and to avoid short circuits.

4.2 Program Development and Uploading

- The Arduino Integrated Development Environment (IDE) was launched on the computer.
- A new program was written to control the 7-segment display and read the push button inputs.
- The correct board type (Arduino Uno) and the appropriate COM port were selected from the Tools menu.
- The program was verified to check for compilation errors before uploading it to the Arduino board.

- After successful compilation, the code was uploaded to the Arduino Uno through the USB connection.

4.3 Program Operation

- The pins connected to the segments of the 7-segment display were configured as output pins, while the push button pins were configured as input pins.
- A variable was initialized to store the current count value.
- The Arduino continuously monitored the state of the push buttons inside the main program loop.
- When the increment button was pressed, the count value increased and the displayed number was updated accordingly.
- When the reset button was pressed, the count returned to zero, and the display showed the number “0”.
- A display control routine was used to activate the correct combination of segments to represent each number on the 7-segment display.

4.4 Coding

```
const int segmentA = 3;
const int segmentB = 2;
const int segmentC = 8;
const int segmentD = 7;
const int segmentE = 6;
const int segmentF = 4;
const int segmentG = 5;

const int PushBut1 = 11;
const int PushBut2 = 12;

int PB1Count = 0;
int PB1State = 0;
int LastPB1State = 0;

int PB2Count = 0;
int PB2State = 0;
```

```
int LastPB2State = 0;

void setup(){
  Serial.begin(9600);

  pinMode(PushBut1, INPUT_PULLUP);
  pinMode(PushBut2, INPUT_PULLUP);

  pinMode(segmentA, OUTPUT);
  pinMode(segmentB, OUTPUT);
  pinMode(segmentC, OUTPUT);
  pinMode(segmentD, OUTPUT);
  pinMode(segmentE, OUTPUT);
  pinMode(segmentF, OUTPUT);
  pinMode(segmentG, OUTPUT);

  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, HIGH);
}

void loop(){
  Serial.println(PB1Count);
  delay(1000);

  PB1State = digitalRead(PushBut1);
  PB2State = digitalRead(PushBut2);

  if(PB1State != LastPB1State){
    if(PB1State == LOW){
      PB1Count++;
    }
    delay(50);
  }

  if(PB2State != LastPB2State){
    if(PB2State == LOW){
      PB1Count = 0;
    }
    delay(50);
  }

  LastPB1State = PB1State;
  LastPB2State = PB2State;

  if(PB1Count == 0){
    r();
    seg0();
  }
```

```
  if(PB1Count == 1){
    r();
    seg1();
  }

  if(PB1Count == 2){
    r();
    seg2();
  }

  if(PB1Count == 3){
    r();
    seg3();
  }

  if(PB1Count == 4){
    r();
    seg4();
  }

  if(PB1Count == 5){
    r();
    seg5();
  }

  if(PB1Count == 6){
    r();
    seg6();
  }

  if(PB1Count == 7){
    r();
    seg7();
  }

  if(PB1Count == 8){
    r();
    seg8();
  }

  if(PB1Count == 9){
    r();
    seg9();
  }
}

void r(){
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  digitalWrite(segmentE, HIGH);
```

```
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, HIGH);
}

void seg0(){
  //0
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
}

void seg1(){
  //1
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
}

void seg2(){
  //2
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentG, LOW);
}

void seg3(){
  //3
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentG, LOW);
}

void seg4(){
  //4
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
}

void seg5(){
  //5
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
```

```
}

void seg6(){
  //6
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
}

void seg7(){
  //7
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentC, LOW);
}

void seg8(){
  //8
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentB, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
}

void seg9(){
  //9
  digitalWrite(segmentA, LOW);
  digitalWrite(segmentC, LOW);
  digitalWrite(segmentD, LOW);
  digitalWrite(segmentE, LOW);
  digitalWrite(segmentF, LOW);
  digitalWrite(segmentG, LOW);
}
```

5.0 DATA COLLECTION

BUTTON PRESSED	COMMAND	NUMBER DISPLAY
-	-	0
Push Button 1	Increment	1
Push Button 1	Increment	2
Push Button 1	Increment	3
Push Button 1	Increment	4
Push Button 1	Increment	5
Push Button 1	Increment	6
Push Button 1	Increment	7
Push Button 1	Increment	8
Push Button 1	Increment	9
Push Button 2	Reset	0

Table 5.1 shows the data collected from button pressed sequence

6.0 DATA ANALYSIS

The data taken during the task is about 7-Segment display functionality based on a digital logic circuit. When a push button is pressed, the input signal is sent to the microcontroller and the output signal is sent based on which 7 LEDs in the 7-Segment display will be turned on. For this task, when the push button 1 is pressed, the number on the 7-Segment display will increase and when the push button 2 is pressed, the number will reset. Based on Table 5.1, the data shows that every time the push button 1 is pressed, the number in the display will increase by 1, which from 0 to 1, and the number will increase again when the push button 1 is pressed again.

Moreover, when the push button 2 is pressed, the program will restart the number from 0, which is shown in Table 5.1 that the display shows number 0 when the push button 2 is being pressed. The reset button is good to be used from restarting the counter or emergency usage.

In conclusion, the result obtained during the task shows how the 7-Segment display works and how the digital logic circuit is applied during the process of doing this task. It also shows the connection of input, output and microcontroller with the functions of push button interactions, through the microcontroller and the display on numbers on 7-Segment display.

7.0 RESULTS

The result for this experiment shows that the starting number displayed by the LCD is 0. When the button is pushed, the number increases by one. It can display numbers from 0 to 9. Once the number reaches 9, we can push the reset button on the Arduino Uno to reset it back to 0 again. Below is the image of the demonstration.

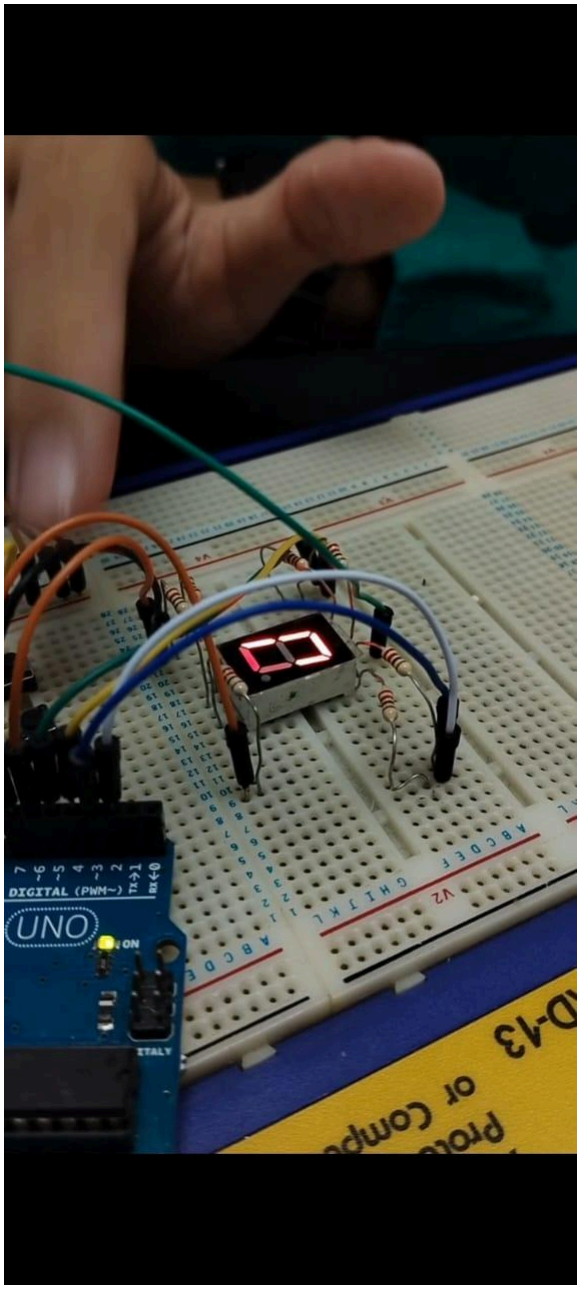


Figure 7.1 Display number 0

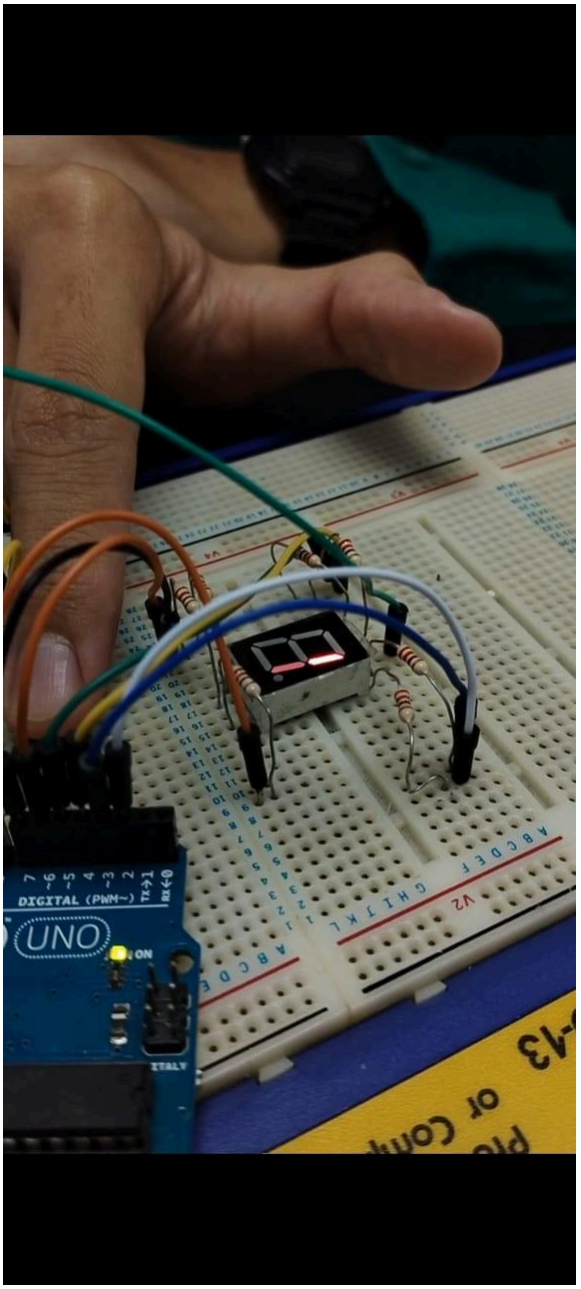


Figure 7.2 Display number 1

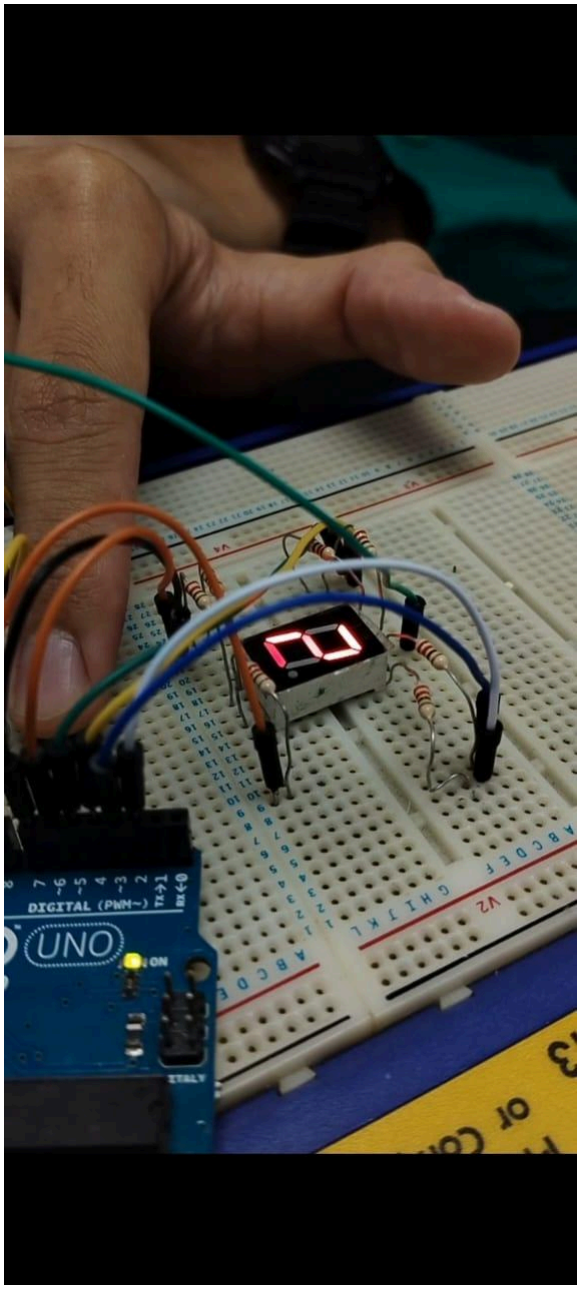


Figure 7.3 Display number 2

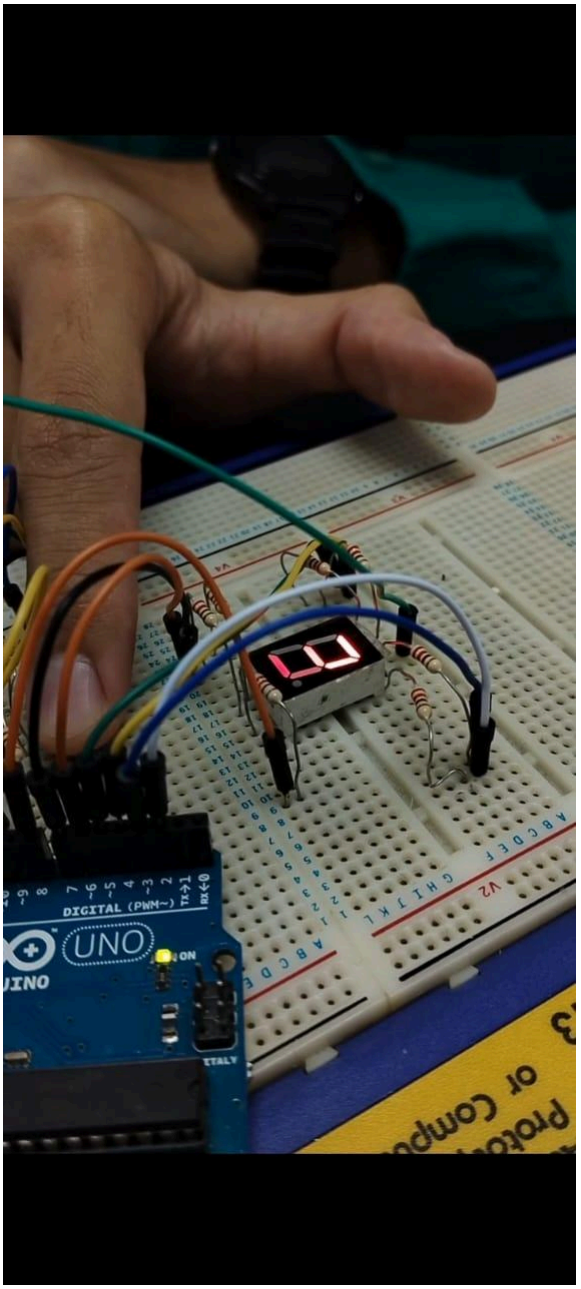


Figure 7.4 Display number 3

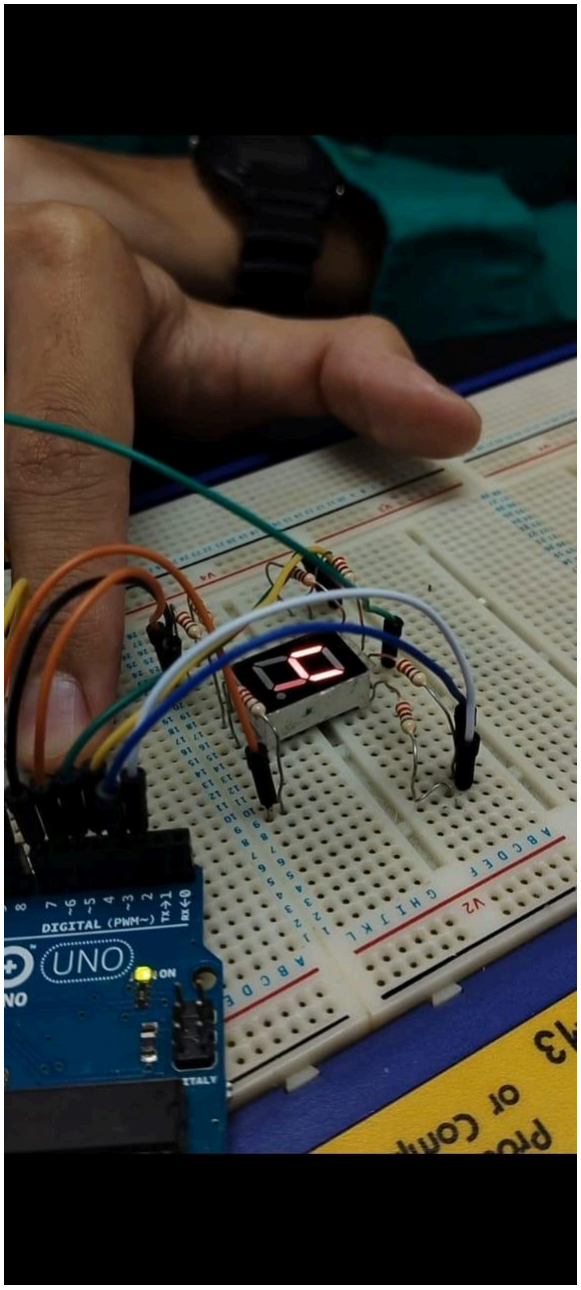


Figure 7.5 Display number 4

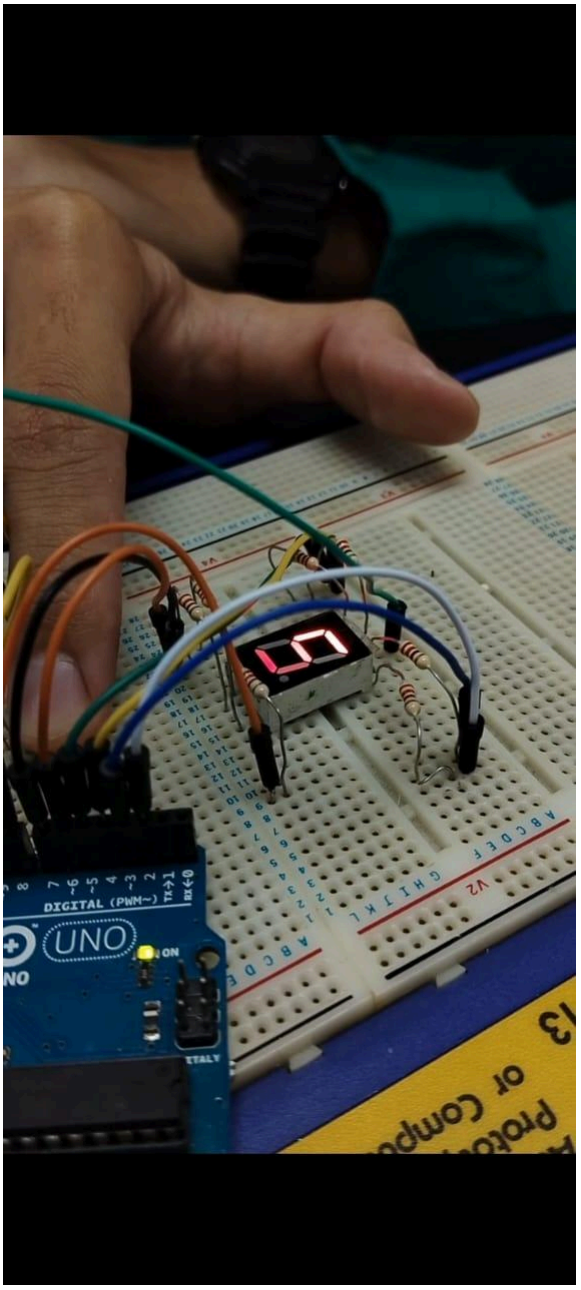


Figure 7.6 Display number 5

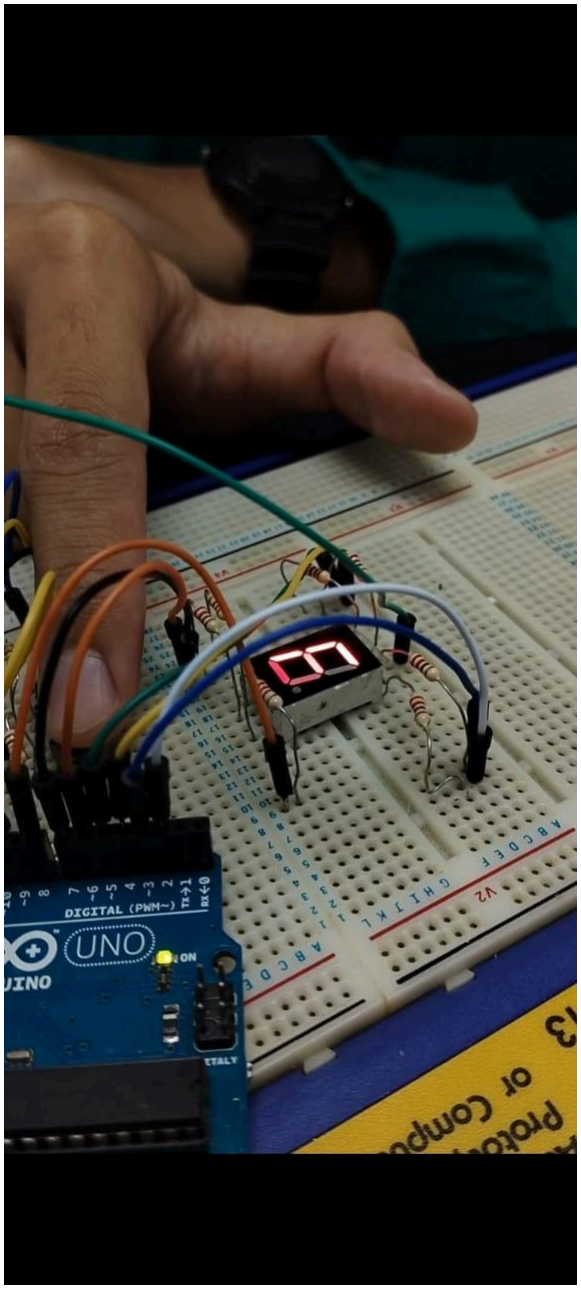


Figure 7.7 Display number 6

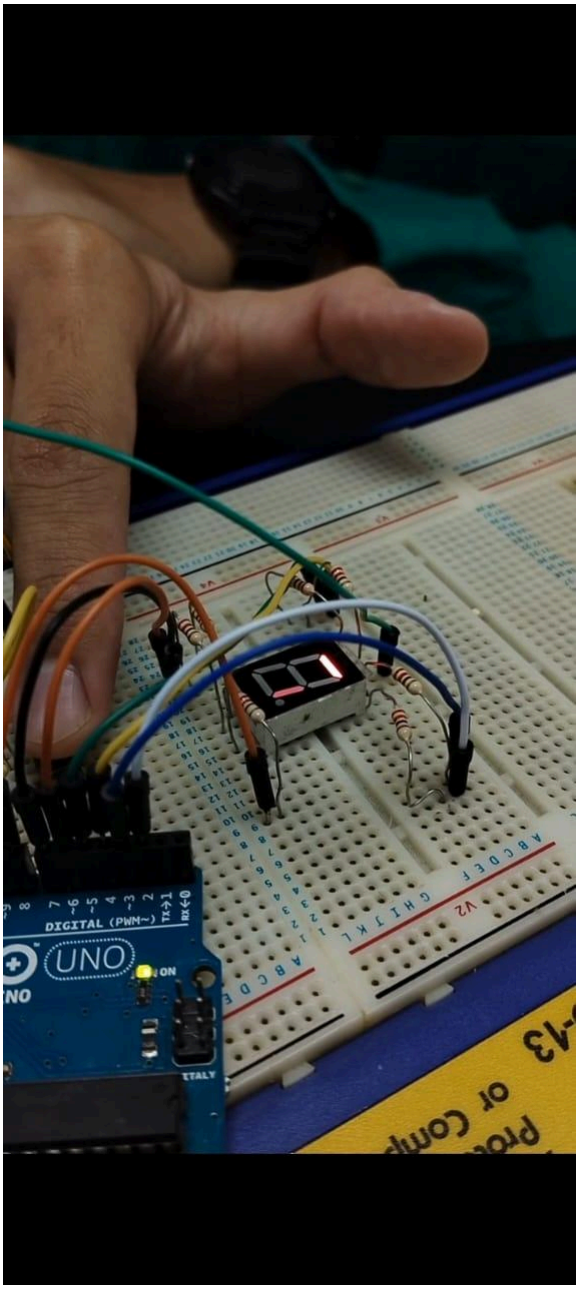


Figure 7.8 Display number 7

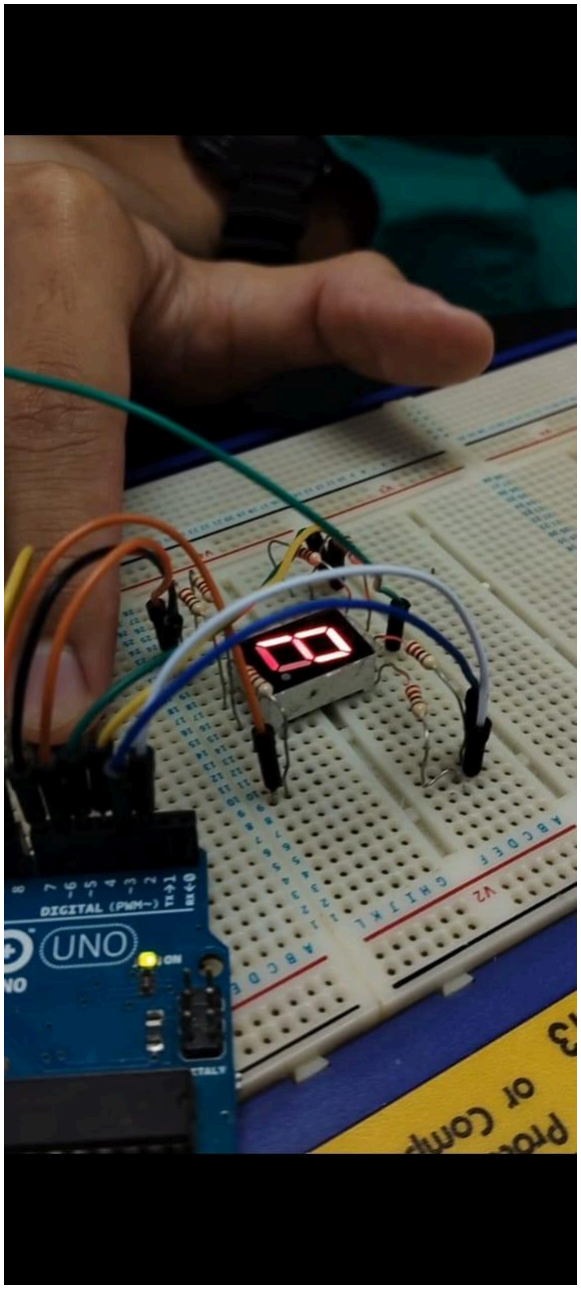


Figure 7.9 Display number 8

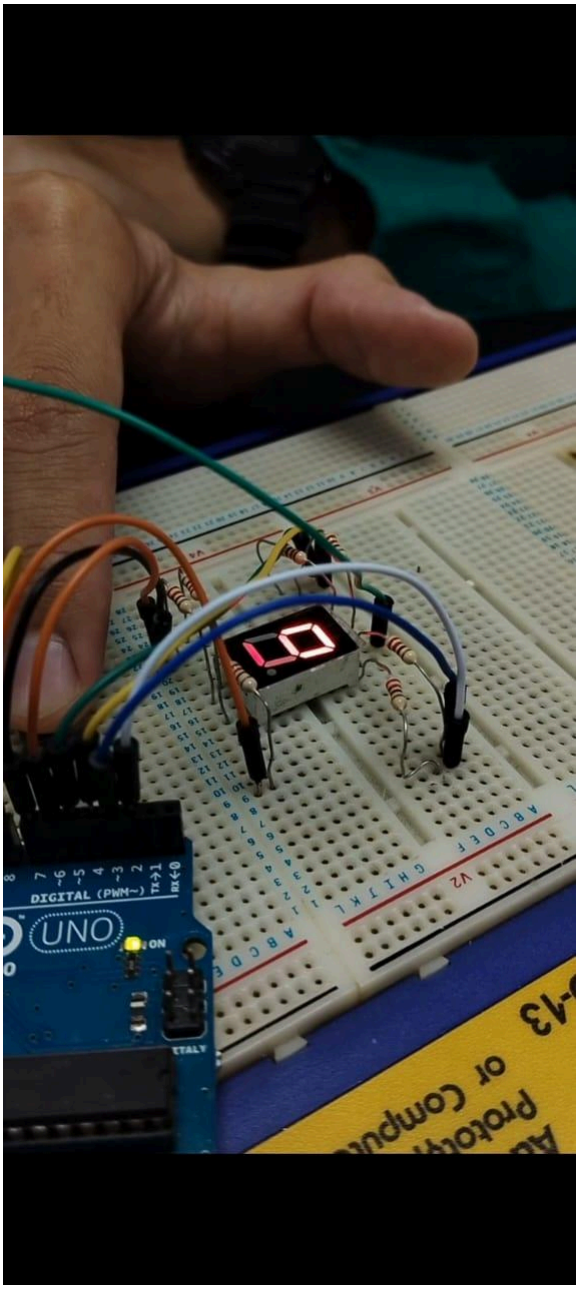


Figure 7.10 Display number 9

8.0 DISCUSSION

This experiment shows that the LCD can be controlled automatically or manually using the Arduino Uno. For automatic control, we can code it exactly how we want. We can change the counting speed by increasing or decreasing the delay time. We can also change the number displayed by controlling each segment; we set it to HIGH for the segments we want to turn on, and LOW otherwise. This allows the counter to go forward or backward. For manual control, we need to add one device: a push button. When the button is not pressed, it creates an open circuit so no current flows through it. When the button is pushed, it completes the circuit, allowing current to flow, and the Arduino will read it as HIGH.

Questions:

How can you interface an I2C LCD with Arduino? Explain the coding principle behind it compared to a 7-segment display and a matrix LED.

To interface the 12C LCD with Arduino, you need to have 4 wires. 1 wire connects with the power source usually using 5V, 1 with ground, 1 with SDA (serial data), lastly with SCL (serial clock), The SDA and SCL need to connect with Analog pin on the Arduino, usually using A4 and A5. The coding principle for both is completely different. For a 12C LCD, we just need to include “LiquidCrystal_I2C” library in our coding. Then we can command like this: lcd.print(“Hello World”);. It is far different with 7 segment lcd as we need to control each segment manually to get what we want to display. For example, to display 1, we need segment B and C on HIGH while the rest are LOW. After that, the coding for the matrix led like 8x8 led is far more complex compared to the 7 segment led and 12C lcd. We need to control all columns and rows manually. It is like controlling all 64 lcd together.

9.0 CONCLUSION

In conclusion, the experiment successfully demonstrated how a 7-segment display can be interfaced with an Arduino Uno microcontroller. The system was able to display numbers and respond to push button inputs for incrementing and resetting the count.

The experiment helped in understanding the relationship between digital logic signals, hardware connections, and software control. It also highlighted the importance of proper circuit design, including current-limiting resistors and pull-up resistors, to ensure stable and safe operation of electronic components.

Overall, the experiment provided practical experience in implementing basic digital logic systems using a microcontroller platform.

10.0 RECOMMENDATIONS

Several improvements can be implemented to enhance the functionality, efficiency, and learning outcomes of this experiment in the future. First, the system can be expanded by using multiple 7-segment displays instead of only one. If there were multiple 7-segment displays, then it could display numbers beyond 9, allowing the user to create counters with two or more digits. For example, the circuit could be modified to count from 00 to 99 or even higher. This improvement would introduce students to techniques such as multiplexing, which is commonly used in digital display systems to control multiple displays using fewer microcontroller pins.

Second, button debouncing techniques should be implemented in the software. When you press a mechanical button, the button's internal electrical contact will bounce several times before settling down. Without proper debouncing, the Arduino may interpret a single press as

multiple inputs, causing the counter to increase unexpectedly. Debouncing can be done through either hardware, such as by adding a capacitor or through software delay.

The project can also be improved by integrating additional display devices such as LCD modules or LED matrix displays. Unlike 7-segment displays, these technologies present more than just numerical data by showing alphanumeric characters, graphics, and symbols. An example would be to use an LCD to show messages such as "COUNT: 05" and other relevant information and therefore making the system more interactive and informative.

Lastly, by including additional control inputs or sensors, the project would have the opportunity to turn into a useful real-world application. For example, when a person approaches or moves past the sensor points it may automatically add counts to the total. Thus, the projects may become useful in counting things like the number of people in a room, counting items in a manufacturing environment, or counting events such as a race.

Overall, these improvements would not only enhance the performance of the system but also provide a deeper understanding of microcontroller interfacing, digital logic implementation, and embedded system design.

11.0 REFERENCES

Watson, E. (n.d.). Sensor Interfacing: A Complete Beginner's Guide, Techniques, and Tips. *Electronics for You – Official Site ElectronicsForU.com*. https://www.electronicsforu.com/technology-trends/learn-electronics/sensor-interfacing

Arduino. (2024). *Debounce*. Arduino Documentation. https://docs.arduino.cc/built-in-examples/digital/Debounce/

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We certify that this report is entirely our own work, except where we have given fully documented references to the work of others, and that the material in this assignment has not previously been submitted for assessment in any formal course of study.

We understand that plagiarism, falsification, and fabrication of data are serious academic offences and may result in disciplinary action under the regulations of International Islamic University Malaysia. Therefore, we take full responsibility for the originality and authenticity of this report.

Signature: 

Name: NOR EFFENDI BIN ANUAR @ ANWAR

Matric Number: 2310519

Signature: 

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI

Matric Number: 2411644

Signature: 

Name: NUR ATIKAH BINTI KAMARUL BAHARIN

Matric Number: 2319846